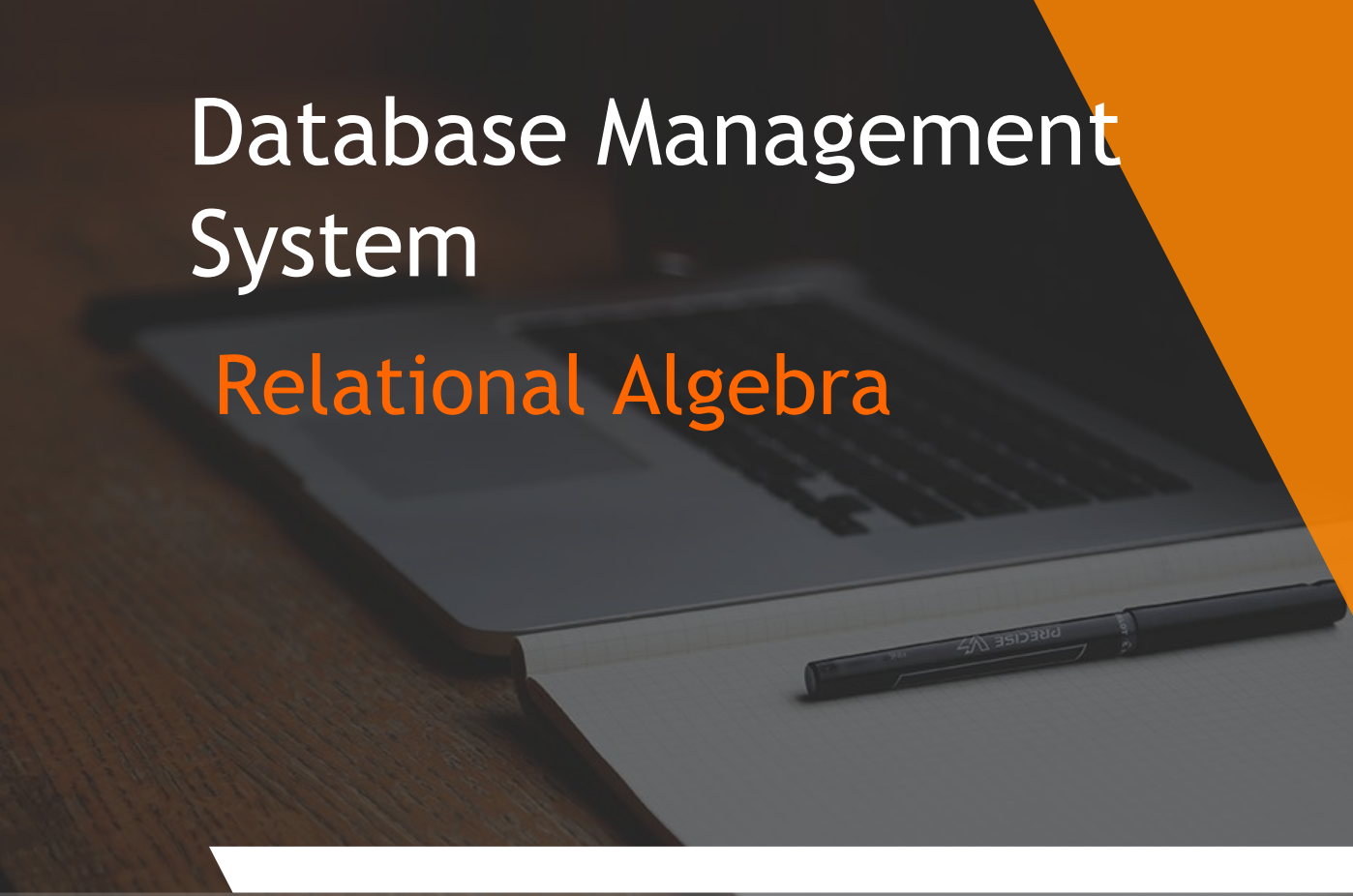
Database Management System

Relational Algebra



Antu Chowdhury
Lecturer
Dept of CSE, EWU

Relational Query Language

- Language in which user requests information from the database.
- Categories of languages
  - Procedural
  - Non-procedural, or declarative **SQL**
- “Pure” languages:
  - Relational algebra
- Pure languages form underlying basis of query languages that people use.

Relational Algebra

- Procedural language
- Six basic operators
  - select: σ
  - project: ∏
  - union: ∪
  - set difference: –
  - Cartesian product: x
  - rename: ρ
- The operators take one or two relations as inputs and produce a new relation as a result.

Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the "Physics" department.

  Query

$$\sigma_{dept\_name="Physics"}(instructor)$$

Result

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select Operation

- We allow comparisons using
  $=, \neq, >, \geq, <, \leq$
  in the selection predicate.
- We can combine several predicates into a larger predicate by using the connectives:
  $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)
- Example:

1. Find the instructors in Physics with a salary greater $90,000, we write:

$$\sigma_{dept\_name=\text{“Physics”} \wedge salary > 90,000}(instructor)$$

Select Operation

In SQL, the SELECT condition is specified in the WHERE clause of a query.For example, the following operation:

$$\sigma_{Dno=4\ AND\ Salary>25000}\ (EMPLOYEE)$$

would do the following SQL query:

SELECT * FROM EMPLOYEE WHERE Dno=4 AND Salary>25000;

Project Operation

- The PROJECT operation denoted by selects certain columns from the table and discards the other columns

$$\Pi_{A_1, A_2, \ldots, A_k}(r)$$

where A_1, A_2 are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- **Duplicate rows removed from result, since relations are sets**

Project Operation

Example: eliminate the *dept_name* attribute of *instructor*

Query: $\Pi_{ID,\ name,\ salary}$ (*instructor*)

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Project Operation

Example:

Relation *r*:

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$$\Pi_{A,C}\,(r)$$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2

Project Operation

In SQL, the PROJECT attribute list is specified in the SELECT clause of a query. For example, the following operation:

$$\pi_{\text{gender, Salary}}(\text{EMPLOYEE})$$

would correspond to the following SQL query:

SELECT DISTINCT gender, salary FROM EMPLOYEE;

Composition of Relational Operations

- The result of a relational-algebra operation is relation and therefore of relational-algebra operations can be composed together into a **relational-algebra expression**.

  Consider the query -- Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept\_name = \text{“Physics”}} (instructor))$$

- IN SQL,

```
SELECT name
FROM instructor
WHERE dept_name=“Physics”;
```

Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, ρ, is provided for that purpose
  - The expression:

    $\rho_x(E)$
  - Returns the result of expression E under the name x
  - Another form of the rename operation:

    $\rho_{x(A1,A2,..An)}(E)$

Emp

emp_id	name	age	addr	sal
1	ABC	33	Pune	5000
2	PQR	36	Mumbai	10000
3	ABC	33	Pune	5000
4	PQR	24	Noida	5400

In SQL: Rename Table

```
SELECT name,age
FROM Emp AS Employee
WHERE age>30;
```

Employee

name	age
ABC	33
PQR	36
ABC	33

In SQL: Rename column

```
SELECT age AS e_age
FROM Emp
WHERE age>30;
```

e_age
33
36
33

Rename Operation

Example: Query to rename the attributes name, age of table "Emp" to e_name, e_age

$$\rho_{(e\_name,\ e\_age)}(Emp)$$

e_name	e_age
ABC	33
PQR	36
ABC	32
PQR	24
LMN	18
UVW	21

Emp

emp_id	name	age	addr	sal
1	ABC	33	Pune	5000
2	PQR	36	Mumbai	10000
3	ABC	32	Delhi	15000
4	PQR	24	Noida	5400
5	LMN	18	Patna	4500
6	UVW	21	Mumbai	400

Example: Query to rename the table name "Emp" to "Employee" and its attributes to e_id, e_name, e_age, e_addr, e_sal

$$\rho_{Employee(e\_id,\ e\_name,\ e\_age,\ e\_addr,\ e\_sal)}(Emp)$$

Rename Operation

- For most queries, we need to apply several relational algebra operations one after the other.
- Either we can write the operations as a single relational algebra expression by nesting the operations, or we can apply one operation at a time and create intermediate result relations.

$$\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno=5}}(\text{EMPLOYEE}))$$

In the latter case, we must give names to the relations that hold the intermediate results.

Rename Operation

- For example, to retrieve the first name, last name, and salary of all employees who work in department number 5, we must apply a SELECT and a PROJECT operation. We can write a single relational algebra expression, also known as an in-line expression, as follows:

$$\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno=5}}(\text{EMPLOYEE}))$$

- Alternatively, we can explicitly show the sequence of operations, giving a name to each intermediate relation, as follows:

$$\text{DEP5\_EMPS} \leftarrow \sigma_{\text{Dno=5}}(\text{EMPLOYEE})$$

$$\text{RESULT} \leftarrow \pi_{\text{Fname, Lname, Salary}}(\text{DEP5\_EMPS})$$

Rename Operation

- Renaming in SQL is accomplished by aliasing using AS, as in the following
- example:

```
SELECT E.Fname AS First_name,
E.Lname AS Last_name,
E.Salary AS Salary
FROM EMPLOYEE AS E
WHERE E.Dno=5
```

- If **only one table** is used in the query and **column names are not ambiguous**, you can omit **E**.
- If more than one table has the **same column name**, you **must** qualify the column.

Union Operation

- The union operation allows us to combine two relations
- Notation: $r \cup s$

For $r \cup s$ to be valid.

1. The relations r and s must be of the same arity. That is, they must have the same number of attributes.
2. The domains of the i_{th} attribute of r and the i_{th} attribute of s must be the same, for all i.

Note that r and s can be either database relations or temporary relations that are the result of relational-algebra expressions.

Example: to find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both

$$\prod_{course\_id} (\sigma_{semester=\text{“Fall”} \wedge year=2009}(section)) \cup \prod_{course\_id} (\sigma_{semester=\text{“Spring”} \wedge year=2010}(section))$$

Union Operation

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Courses offered in either Fall 2009, Spring 2010 or both semesters.

Union Operation

$$\Pi_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2009}(section)) \cup \Pi_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2010}(section))$$

In SQL,

```
SELECT course_id FROM section
WHERE semester=“Fall” AND year=2009
UNION
SELECT course_id FROM section
WHERE semester=“Fall” AND year=2010
```

Union Operation

Example: Consider the following two relations:

A	B
α	1
α	2
β	1

r

C	D
α	2
β	3

s

● r ∪ s:

A	B
α	1
α	2
β	1
β	3

Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$

Assume:

r, s have the *same arity*

attributes of r and s are compatible

Example: Find the set of all courses taught in both the Fall 2009 and the Spring 2010 semesters.

$$\prod_{course\_id} (\sigma_{semester=\text{“Fall”} \wedge year=2009} (section)) \cap \prod_{course\_id} (\sigma_{semester=\text{“Spring”} \wedge year=2010} (section))$$

Intersection Operation

Example: Consider the following two relations:

A	B
α	1
α	2
β	1

r

C	D
α	2
β	3

S

A	B
α	2

- *We also can write: r ∩ s= r-(r-s)*

Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
  - r and s must have the same arity
  - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\Pi_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2009}(section)) - \Pi_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2010}(section))$$

Difference Operation

$$\prod_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2009}(section)) - \prod_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2010}(section))$$

```
SELECT course_id FROM section
WHERE semester=“Fall” AND year=2009
MINUS
SELECT course_id FROM section
WHERE semester=“Fall” AND year=2010
```

Difference Operation

Relations *r*, *s*:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- ***r* − *s*:**

A	B
α	1
β	1

Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by ← and works like assignment in a programming language.
- Example: Find all instructor in the “Physics” and Music department.

$$Physics \leftarrow \sigma_{dept\_name=\text{“Physics”}}(instructor)$$
$$Music \leftarrow \sigma_{dept\_name=\text{“Music”}}(instructor)$$
$$Physics \cup Music$$

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query

Cartesian-product Operation

- The Cartesian-product operation (denoted by X) allows us to combine information from any two relations.
- Example: the Cartesian product of the relations *instructor* and t*eaches* is written as:

  instructor X *teaches*
- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation (see next slide)
- Since the instructor *ID* appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.

  instructor.ID

  teaches.ID

Cartesian-product Operation

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 6.1 The *instructor* relation.

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Figure 6.7 The *teaches* relation.

Cartesian-product Operation

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Cartesian-product Operation

Suppose that we want to find the names of all instructors in the Physics department together with the *course_id* of all courses they taught. We need the information in both the *instructor* relation and the *teaches* relation to do so. If we write:

$$\sigma_{dept\_name = \text{“Physics”}}(instructor \times teaches)$$

then the result is the relation in Figure 6.9.

Cartesian-product Operation

inst.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
22222	Einstein	Physics	95000	10101	CS-437	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
22222	Einstein	Physics	95000	32343	HIS-351	1	Spring	2010
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
33456	Gold	Physics	87000	10101	CS-437	1	Fall	2009
33456	Gold	Physics	87000	10101	CS-315	1	Spring	2010
33456	Gold	Physics	87000	12121	FIN-201	1	Spring	2010
33456	Gold	Physics	87000	15151	MU-199	1	Spring	2010
33456	Gold	Physics	87000	22222	PHY-101	1	Fall	2009
33456	Gold	Physics	87000	32343	HIS-351	1	Spring	2010
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Figure 6.9 Result of $\sigma_{dept\_name = \text{"Physics"}}(instructor \times teaches)$.

Cartesian-product Operation

Since the Cartesian-product operation associates *every* tuple of *instructor* with every tuple of *teaches*, we know that if a an instructor is in the Physics department, and has taught a course (as recorded in the *teaches* relation), then there is some tuple in $\sigma_{dept\_name = \text{"Physics"}}(instructor \times teaches)$ that contains his name, and which satisfies $instructor.ID = teaches.ID$. So, if we write:

$$\sigma_{instructor.ID = teaches.ID}(\sigma_{dept\_name = \text{"Physics"}}(instructor \times teaches))$$

we get only those tuples of *instructor* × *teaches* that pertain to instructors in Physics and the courses that they taught.

Finally, since we only want the names of all instructors in the Physics department together with the *course_id* of all courses they taught, we do a projection:

$$\Pi_{name,\ course\_id}\ (\sigma_{instructor.ID = teaches.ID}(\sigma_{dept\_name = \text{"Physics"}}(instructor \times teaches)))$$

Cartesian-product Operation

The result of this expression, shown in Figure 6.10, is the correct answer to our query. Observe that although instructor Gold is in the Physics department, he does not teach any course (as recorded in the *teaches* relation), and therefore does not appear in the result.

name	*course_id*
Einstein	PHY-101

Figure 6.10 Result of $\Pi_{name,\ course\_id}\ (\sigma_{instructor.ID = teaches.ID}(\sigma_{dept\_name = \text{“Physics”}}(instructor \times teaches)))$.

Cartesian-product Operation

Note that there is often more than one way to write a query in relational algebra. Consider the following query:

$$\Pi_{name,\ course\_id}\ (\sigma_{instructor.ID = teaches.ID}((\sigma_{dept\_name = \text{“Physics”}}(instructor)) \times\ teaches))$$

Note the subtle difference between the two queries: in the query above, the selection that restricts *dept_name* to Physics is applied to *instructor*, and the Cartesian product is applied subsequently; in contrast, the Cartesian product was applied before the selection in the earlier query. However, the two queries are **equivalent**; that is, they give the same result on any database.

Cartesian-product Operation

- Relations *r*, *s*:

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

- *r* x *s*:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Cartesian-product Operation

In SQL,

```
SELECT
   instructor.*,
   teaches.*
FROM
   instructor
CROSS JOIN
   teaches;
```

Join Operation

- The Cartesian-Product

  instructor X *teaches*

  associates every tuple of instructor with every tuple of teaches. Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of "*instructor* X *teaches* " that pertain to instructors and the courses that they taught, we write:

  $\sigma_{instructor.id = teaches.id}$ (*instructor* x *teaches*))

  We get only those tuples of "*instructor* X *teaches*" that pertain to instructors and the courses that they taught.

The result of this expression, shown in the next slide

Join Operation

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Join Operation

It's basically a **Cartesian Product followed by a selection.**

🧩 General Definition:

$$R \bowtie_\theta S = \sigma_\theta(R \times S)$$

where

- `R` and `S` are relations,
- `θ` (theta) is a condition,
- `×` is Cartesian product,
- `σ_θ` is the selection operator.

So:

👉 **Join = Cartesian Product + Selection Condition**

Instructor

ID	Name	Dept
1	Alice	CSE
2	Bob	EEE

Teaches

ID	Course
1	DBMS
3	AI

(a) Cartesian Product

Instructor×Teaches

(b) Theta Join (condition: Instructor.ID = Teaches.ID)

Instructor⋈ Instructor.ID=Teaches.IDTeaches

Join Operation

A	B
1	2
4	5
7	2

⋈

B	C
2	3
5	6
2	8

=

A	B	C
1	2	3
4	5	6
7	2	8
1	2	8
7	2	3

Join Operation

Example Theta join

Suppose a customer wants to buy a car and boat, but doesn't want 2 spend more money for the boat than for the car Ie CAR >= Boat Price

Car Model	Car Price
Car A	20000
Car B	30000
Car C	50000

Boat model	Boat Price
Boat 1	10000
Boat 2	40000
Boat 3	60000

Car Model	Car Price	Boat Model	Boat Price
Car A	20000	Boat 1	10000
Car B	30000	Boat 1	10000
Car C	50000	Boat 1	10000
Car C	50000	Coat 2	40000

Join Operation

```
SELECT instructor.*, teaches.*
FROM instructor
JOIN
teaches
ON instructor.ID = teaches.ID
```

```
SELECT instructor.*, teaches.*
FROM instructor , teaches
WHERE instructor.ID = teaches.ID
```

Join Operation

Banking Example

branch (branch_name, branch_city, assets)

customer (customer_name, customer_street, customer_city)

account (account_number, branch_name, balance)

loan (loan_number, branch_name, amount)

depositor (customer_name, account_number)

borrower (customer_name, loan_number)

Example Queries

- Find all loans of over $1200

$$\sigma_{amount > 1200}(loan)$$

- Find the loan number for each loan of an amount greater than $1200

$$\Pi_{loan\_number}(\sigma_{amount > 1200}(loan))$$

Example Queries

- Find the names of all customers who have a loan, an account, or both, from the bank

$$\Pi_{customer\_name}(borrower) \cup \Pi_{customer\_name}(depositor)$$

- Find the names of all customers who have a loan at the Perryridge branch.

$$\Pi_{customer\_name}(\sigma_{branch\_name=\text{“Perryridge”}}(\sigma_{borrower.loan\_number = loan.loan\_number}(borrower \times loan)))$$

Or,

$$\Pi_{customer\_name}(\sigma_{branch\_name=\text{“Perryridge”}}((borrower \bowtie loan)))$$

Example Queries

- Find the names of all customers who have a loan at the Perryridge branch but do not have an account at any branch of the bank.

$$\prod_{customer\_name} (\sigma_{branch\_name = \text{“Perryridge”}} (\sigma_{borrower.loan\_number = loan.loan\_number}(borrower \times loan))) - \prod_{customer\_name}(depositor)$$

$$\prod_{customer\_name} (\sigma_{branch\_name = \text{“Perryridge”}} ((borrower \bowtie loan))) - \prod_{customer\_name}(depositor)$$

Example Queries

- Find the names of all customers who have a loan at the Perryridge branch.

  - Query 1

$$\prod_{customer\_name} (\sigma_{branch\_name = \text{“Perryridge”}} (\sigma_{borrower.loan\_number = loan.loan\_number} (borrower \times loan)))$$

  - Query 2

$$\prod_{customer\_name}(((\sigma_{branch\_name = \text{“Perryridge”}} (loan)) \bowtie borrower))$$

Example Queries

- Find the largest account balance
  - Strategy:
    - Find those balances that are *not* the largest
      - Rename *account* relation as *d* so that we can compare each account balance with all others
    - Use set difference to find those account balances that were *not* found in the earlier step.

$$\Pi_{balance}(account) - \Pi_{account.balance}(\sigma_{account.balance < d.balance}(account \times \rho_{d}(account)))$$

Example Queries

- Find the name of all customers who have a loan at the bank and the loan amount

$$\Pi_{customer\_name,\ loan\_number,\ amount}\ (borrower \bowtie\ loan)$$

- Find the names of all customers who have a loan and an account at bank.

$$\Pi_{customer\_name}\ (borrower) \cap \Pi_{customer\_name}\ (depositor)$$

Example Queries

- Find all customers who have an account from at both the “Downtown” and the Uptown” branches.

$$\Pi_{customer\_name}(\sigma_{branch\_name = \text{“Downtown”}}(depositor \bowtie account)) \cap$$

$$\Pi_{customer\_name}(\sigma_{branch\_name = \text{“Uptown”}}(depositor \bowtie account))$$

Example Queries

b) Consider the following relational database schema

passenger (**pid**, pname, pgender, pcity)

agency (**aid**, aname, acity)

flight (**fid**, fdate, time, src, dest)

booking (**pid**, **aid**, **fid**, fdate)

Answer the following questions using relational algebra

i. Get the details about all flights from Dhaka to Cox's Bazar.

ii. Find the passenger names for passengers who have bookings on at least one flight.

iii. Find the passenger names for those who do not have any bookings in any flights.

iv. Find the details of all male passengers who are associated with ShareTrip agency.

Example Queries

b) Consider the following relational database schema

passenger (**pid**, pname, pgender, pcity)

agency (**aid**, aname, acity)

flight (**fid**, fdate, time, src, dest)

booking (**pid**, **aid**, **fid**, fdate)

More Queries:

- **Find only the flight numbers for passenger with pid 123 for flights to Chennai before 06/11/2020.**
- **Find the agency names for agencies that located in the same city as passenger with passenger id 123.**
- **Get the details of flights that are scheduled on both dates 01/12/2020 and 02/12/2020 at 16:00 hours.**

Example Queries

b) Consider the following relational database schema

passenger (**pid**, pname, pgender, pcity)

agency (**aid**, aname, acity)

flight (**fid**, fdate, time, src, dest)

booking (**pid**, **aid**, **fid**, fdate)

More Queries:

- **Get the details of flights that are scheduled on either of the dates 01/12/2020 or 02/12/2020 or both at 16:00 hours.**

- **Find the agency names for agencies who do not have any bookings for passenger with id 123.**

Aggregate Operation

- An **aggregate function** takes a collection of values and returns a single value as a result.

  avg: average value
  min: minimum value
  max: maximum value
  sum: sum of values
  count: number of values

- **Aggregate operation** in relational algebra

$$G_1, G_2, \ldots, G_n \; g \; F_1(A_1), F_2(A_2), \ldots, F_n(A_n) \; (E)$$

  - E is any relational-algebra expression
  - $G_1, G_2 \ldots, G_n$ is a list of attributes on which to group (can be empty)
  - Each F_i is an aggregate function
  - Each A_i is an attribute name

Aggregate Operation

- Relation *r*:

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

$$g_{\text{sum(c)}}(\text{r})$$

sum-C
27

No grouping

Aggregate Operation

- Relation *account* grouped by *branch-name*:

branch-name	account-number	balance
Perryridge	A-102	400
Perryridge	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

$$branch\text{-}name\ \mathcal{G}\ sum(balance)\ (account)$$

branch-name	sum(balance)
Perryridge	1300
Brighton	1500
Redwood	700

Aggregate Operation

Result of aggregation does not have a name

- Can use rename operation to give it a name
- For convenience, we permit renaming as part of aggregate operation

$$branch\text{-}name\ \mathcal{G}\ sum(balance)\ as\ sum\text{-}balance\ (account)$$

Aggregate Operation

Sample Input Table: Employee

Dept	EmpID	Salary
HR	1	50000
HR	2	55000
Engineering	3	70000
Engineering	4	80000
Marketing	5	60000
Marketing	6	65000

Average Salary per Department

Relational Algebra Expression:

$$\text{Dept}\ \mathcal{g}\ \text{AVG(Salary)(Employee)}$$

Dept	AVG(Salary)
HR	52500
Engineering	75000
Marketing	62500

Aggregate Operation

Total Salary and Count of Employees per Department

Algebra Expression:

$$\text{Dept}\ \mathcal{g}\ \text{SUM(Salary),COUNT(EmpID)(Employee)}$$

Dept	SUM(Salary)	COUNT(EmpID)
HR	105000	2
Engineering	150000	2
Marketing	125000	2

Example Queries

Consider the following relational database schema

user(**user_id**, name,email, passward, gender)

post(**post_id**, user_id(FK), content, likes)

comment(**ID**, post_id(FK), user_id(FK), time, content)

friend(**ID**, status)

Answer the following questions using relational algebra

i. Find the post contents along with the user ID which post does not have any comment.

ii. Get the details about all female users who have an ID in between 2 to 15.

iii. Find the details of all users who have at least one friend and have more than 5 posts.

Example Queries

Consider the following relational database schema

Suppliers (sid : integer, sname : string, address : string , area : string)
Parts (pid : integer, pname : string, color : string)
Catalog (sid : integer, pid : integer, cost : real(number), amount : integer)

Answer the following questions using relational algebra

i. Get the details about all "Parts" available in "Dhaka" area.

ii. Find the "Suppliers" (sid) who is selling "Green" coloured "Wool" and have more than 500 unit stock.

iii. Find the output for the following:

Π (amount, cost) (σ (pid = (Π pid (σ (color = 'red')parts)) Catalog)

[Just write a sentence what will the query find.
Example : All supplier with that sells red parts]

Example Queries

Consider the following relational database schema:
Passenger (p_id, name, password, location)
Driver (d_id, name, password, v_id)
Vehicles (v_id, d_id, name, status)
Trips (t_id, p_id, date, time, route, cost)

Answer the following questions using relational algebra:

i) Find all the information of trips between 1000 to 5000 tk on 29th February.
ii) Find the names and vehicle id of drivers who had vehicles whose status are "Unfit".
iii) Find the output for the following:

Π (t_id, date, route) (σ (cost > 3000 ^ route = (Π location(Passenger))) trips)

Note: Date format is YYYY-MM-DD. And for the last question, write a sentence describing what
the query will return.

Example: All passenger names whose location is Dhaka.

Example Queries

Consider the following Schema:

User (UserId, Name, Password)
Patent (PatentNo, PatentTitle, PatentContents, Diagrams, UserId (Foreign Key))
Bid (BidNo, UserId (Foreign Key), PatentNo (Foreign Key), StartBid, FinalSellingPrice)

Give an expression in the relational algebra to express each of the following queries:

i) Find all the bids that started at a price greater than 3000 and finally sold at less than 15,000.

ii) Find the patent no. and patent titles for all the users who sold their patents at a price greater than 2000.

Find the output for the following:

Π(PatentNo, PatentContents) (σ(StartBid > 3000 ^ PatentNo = (Π PatentNo(Patent))) Bid)

Write a sentence describing what the query will return.

Example: All patent holders whose names are John.

Example Queries

Consider the following relational database of a college.

Student(RollNumber, StudentName, Address)

Teachers(TeacherID, TecherName, TeachingSubject)

College(RollNumber, TeacherID)

Write SQL or Relational algebra expression for the following requests.

a. Find the name of Students who live in Lalitpur.

b. Find the name of teacher who teaches Database Management System subject.

c. Find the name of teacher who teaches Computer Organization subject to John Smith student.

Example Queries

Use these relations to solve in relational algebra de queries that follow.

employees(*eid, name, salary, did, m_did*)
projects(*pid, description*)
works_on(*eid, pid, hours*)
departments(*did, location*)

1. List the name of the project that have employees from the systems department working less than 5 hours. Pid is also the name of the project.

2. List the name of employees with a salary greater than their manager's salary.

4. List the name of employees making more than $100,000 and working on zero projects

Example Queries

Consider the relational database below, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries :

Employee (pname, street, city)

Works (pname, cname, salary)

Company (cname, city)

Manages (pname, mgrname)

1. Find the names of all employees who work for First Bank Corporation.
2. Find the names and cities of residence of all employees who work for First Bank Corporation.
3. Find the names, street address, and cities of residence of all employees who work for First Bank Corporation and earn more than $10,000 per annum.
4. Find the names of all employees in this database who live in the same city as the company for which they work.
5. Find the names of all employees who live in the same city and on the same street as do their managers.
6. Find the names of all employees in this database who do not work for First Bank Corporation.
7. Find the names of all employees who earn more than every employee of Small Bank Corporation.

Example Queries